Feasibility Study and Implementation Plan

Travel Itinerary and Expense Organizer (MySQL Workbench implementation)

Prepared by: Team (add names)

Date: 23 October 2026 | **Revision 2:** 26 October 2026

Database: `travel_itinerary_db` (version 2, extended)

Target server: MySQL 8.4 LTS (also 8.0.16 or newer)

> **Revision 2 (26 October 2026).** The scope has been extended beyond the original base schema with two add-on scripts: the **transport module** (script 03) and the **smart layer** (script 05). The final database now has 32 tables, 41 foreign keys, 31 triggers and 4 views. In addition, every script has now been executed on a live MySQL 8.4.13 server and all acceptance counts, rule tests and procedures were verified (see section 8).

1. Project summary

The project turns our approved ER diagram into a working relational database. The system lets travellers plan trips day by day, track bookings and expenses, split costs among group members, and reserve hotel rooms with payments and reviews. The database is built in layers: a base schema (script 01) plus a demo/verification script (02), a transport module (03) and a smart layer (05) for auditing and budget alerts.

Item	Value
Entities (tables)	32 — 21 core (people, catalogue, trip planning, money, community, hotel module and lookup tables) + 9 transport + 2 smart layer
Relationships (foreign keys)	41, each with an explicit `ON DELETE` / `ON UPDATE` rule
Special modelling	IS-A (hotels is a subtype of places; `transport_bookings` is a supertype of five mode-specific tables), 3 weak entities (hotel_payments, hotel_reviews, transport_passengers), composite foreign key (rooms to room_types)
Business rules in the database	31 triggers (24 BEFORE triggers that reject bad rows, 7 AFTER triggers for the audit trail and budget alerts), 4 views, 2 stored procedures, 30+ CHECK / UNIQUE constraints
Sample data	399 rows (312 core + 87 transport) across all populated tables, realistic Indian travel scenario in INR; the audit and alert tables start empty
Deliverables	`01_travel_itinerary_v2_full.sql` (schema, triggers, procedures, view, data), `02_demo_queries.sql` (12 queries, 14 rule tests, procedure demo), `03_transport_module.sql` (transport module), `05_smart_features.sql` (audit trail, budget alerts, dashboards)

Objectives

1. Create every table exactly as drawn in the ER diagram, with the same keys, constraints and referential actions.
2. Load meaningful sample data that exercises every relationship.
3. Prove the design works: queries return correct answers and invalid data is rejected by the database itself.
4. Document the technology choice, plan, team split and risks (this document).

2. Feasibility analysis

2.1 Technical feasibility: feasible

Every feature of the diagram has a native MySQL equivalent: composite primary and foreign keys, ENUM, DECIMAL money types, CHECK constraints (enforced from MySQL 8.0.16), BEFORE triggers with `SIGNAL` for business rules, AFTER triggers for audit and alerting, views for derived values, and transactions with `SELECT ... FOR UPDATE` for safe booking.

Derived attributes (nights, total cost, amount paid, balance due, actual spend, remaining budget) are not stored. They are computed in the views or in queries, which keeps the schema normalised.

Rules that SQL constraints cannot express (no overlapping activities in a day, no double-booked room, only completed stays can be reviewed, expense must belong to the same trip, a transport carrier must suit the mode, an itinerary item must belong to the same trip) are implemented as triggers.

The one rule that cannot be enforced declaratively, that a trip owner is also a member with role `owner`, is handled by the `sp_create_trip` transaction and checked by an integrity query (Q12).

Risk level: low. All techniques are covered in a standard DBMS course.

2.2 Operational feasibility: feasible

The team only needs a laptop with MySQL Server and MySQL Workbench. Setup takes under 30 minutes. The scripts are re-runnable (they drop and rebuild their own objects), so anyone can restore a clean state before a demo.

2.3 Economic feasibility: feasible (cost: Rs 0)

MySQL Community Edition, MySQL Workbench and (optionally) Python with Streamlit are free and open source. No paid cloud service is needed.

2.4 Schedule feasibility: feasible

The core database is already scripted. Remaining work is installation, execution on the team's own machines, screenshots, EER diagram export and report writing. A 10-day plan is given in section 7; the critical path is about 4 days.

2.5 Data and legal feasibility: feasible

All sample data is synthetic: fictional people, fictional hotels, `.example` e-mail domains and fake phone numbers. No real personal data is used.

Passwords in the demo are stored as SHA2 hashes only to avoid plain text. A real application must use bcrypt or argon2 with a salt; this is stated as a limitation.

Card numbers are never stored; only a transaction reference is kept.

2.6 Risk register

Risk	Likelihood	Impact	Mitigation
Server and Workbench version mismatch	Medium	Medium	Use MySQL 8.4 LTS; run the scripts in Workbench; check the connection before the demo (see 3.1)
CHECK constraints ignored on very old MySQL	Low	High	Require 8.0.16 or newer; the quick-check query at the end of each script confirms the object counts
Trigger creation blocked by privileges or binary logging	Low	Medium	Use the root account locally; if error 1419 appears, run `SET GLOBAL log_bin_trust_function_creators = 1;`
Workbench stops a script at the first error	Medium	Low	Section C of the demo file is run statement by statement and wrapped in a transaction that is rolled back
A teammate edits the schema separately and versions diverge	Medium	Medium	One shared GitHub repository; only the schema owner merges changes to script 01
Data entry error in sample data	Low	Low	Sample data was dry-run against all constraints and trigger rules, then loaded on MySQL 8.4.13 (section 8)

Overall verdict: the project is technically, operationally, economically and schedule-wise feasible. **Recommendation: proceed.**

3. Technology stack

3.1 Core stack (required)

Layer	Choice	Why
Database server	MySQL Community Server 8.4 LTS	MySQL 8.0 reached end of life in April 2026, so 8.4 LTS is the supported long-term release. The scripts also work on 8.0.16 or newer. Verified on 8.4.13.
Client and modelling	MySQL Workbench	Runs SQL scripts, shows result grids, and generates the EER diagram. If the EER editor is missing in your build, install classic Workbench 8.0.x for diagramming only.
Language	SQL (MySQL dialect)	DDL (CREATE), DML (INSERT, UPDATE, DELETE), DQL (SELECT), TCL (START TRANSACTION, COMMIT, ROLLBACK) and the stored-program language (triggers, procedures)
Storage engine	InnoDB	Required for foreign keys, transactions and row locking
Character set	utf8mb4	Full Unicode, safe for names and symbols
Version control	Git and GitHub	Shared repository, one commit per task

3.2 Optional demo layer (only if time permits)

A small web front end makes the demo more impressive but is not needed for the Workbench submission.

Package	Purpose
Python 3.11 or newer	Application language
streamlit	Quick web UI: trip dashboard, itinerary table, expense chart
mysql-connector-python	Official driver to run queries and stored procedures
pandas	Turn query results into tables and summaries
plotly	Budget-vs-actual and category spend charts
python-dotenv	Keep database credentials out of the code

Install with: `pip install streamlit mysql-connector-python pandas plotly python-dotenv`. Keep the app read-mostly: it calls the same queries and `sp_book_room` / `sp_create_trip`, so all business rules stay inside the database.

3.3 Design decisions worth explaining in the viva

Decision	Reason
`DECIMAL(10,2)` for money	Exact arithmetic; FLOAT would cause rounding errors
ENUM for statuses and roles	Restricts values to a fixed list, cheap to store
`rate_per_night` stored on the reservation	A price snapshot: later changes to the room type's base rate must not change past bookings
`hotels.hotel_id` is both primary key and foreign key to `places`	Implements IS-A with a shared key
`transport_bookings` supertype with five subtype tables	Reuses the same IS-A pattern for flights, trains, cabs, buses and metro; a shared `transport_id` is the primary key of both the supertype and each subtype
`hotel_payments` key is (reservation_id, payment_no)	Weak entity: a payment has no meaning without its reservation
`rooms` has a composite FK (room_type_id, hotel_id)	Guarantees a room's type belongs to the same hotel
Triggers instead of application code	The rule holds no matter which tool inserts the data
BEFORE triggers reject, AFTER triggers react	Validation must run before the row is written; the audit trail and budget alerts run after the row is saved and write a second record
ON DELETE CASCADE for owned children, RESTRICT for shared master data, SET NULL for optional links	Deleting a trip removes its days and items, but a user who owns trips or a category in use cannot be deleted

4. Data design

4.1 Tables by module

Module	Tables
People	users
Catalogue	destinations, places, activity_categories, expense_categories, amenities
Trip planning	trips, trip_members, itinerary_days, itinerary_items, bookings
Money	expenses
Community	reviews, saved_destinations
Hotel module	hotels, hotel_amenities, room_types, rooms, hotel_reservations, hotel_payments, hotel_reviews
Transport module (script 03)	transport_bookings, flight_details, train_details, cab_details, bus_details, metro_details, transport_passengers, carriers, stations
Smart layer (script 05)	audit_log, trip_alerts

4.2 Normalisation

All tables are in third normal form: every non-key attribute depends on the key, the whole key and nothing but the key. Lookup values (categories, amenities, carriers, stations) live in their own tables, many-to-many links use junction tables (trip_members, hotel_amenities, reviews, saved_destinations), and computed values are never stored.

4.3 Where each rule is enforced

Mechanism	Examples
PRIMARY KEY / UNIQUE	unique e-mail; one booking per item; one day per date in a trip; no duplicate room number in a hotel; unique confirmation code; unique carrier name and station code
FOREIGN KEY	41 links with CASCADE, RESTRICT or SET NULL
CHECK	budget at least 0; end date not before start date; rating 1 to 5; payment amount above 0; check-out after check-in; transport arrival after departure; fare at least 0
BEFORE triggers (24)	no overlapping timed items in a day; day_date equals start_date plus day_number minus 1; expense item belongs to the same trip; hotel row must point at a Hotel place; reservation checks (capacity, maintenance, trip dates, booker role, double booking); reviews only for completed stays; transport carrier type must suit the mode; origin and destination stations must match the mode and differ; a transport subtype row must match its booking's mode; only a trip owner or editor may book transport
AFTER triggers (7)	audit trail on trips, hotel_reservations, transport_bookings and expenses (update and delete); budget alerts written to trip_alerts when spending reaches 80 percent of, or exceeds, the trip budget
Transactions	sp_create_trip (trip, owner membership and daily rows together); sp_book_room (row lock, insert, confirmation code)
Integrity query	Q12 confirms every trip owner is a member with role owner

4.4 Transport module and smart layer (scripts 03 and 05)

Transport module (03). Handles flights, trains, cabs, buses and metro using the same IS-A pattern as hotels. `transport_bookings` is the supertype (one row per leg: trip, carrier, times, fare, status) and `flight_details`, `train_details`, `cab_details`, `bus_details` and `metro_details` are subtypes that share `transport_id` as both primary key and foreign key. `transport_passengers` is a weak entity keyed by (`transport_id`, `passenger_no`), and `carriers` and `stations` are lookup tables (airlines, railways, cab and bus operators, metro operators; airports, railway stations, bus stands and metro stations). Public transport is train + bus + metro; cabs are the only mode without stations. The module adds 9 tables, 13 foreign keys, 12 BEFORE triggers and one view (`v_trip_transport`).

Smart layer (05). Adds a generic change history and a budget-alert system. `audit_log` records who changed what, from which value to which value; `trip_alerts` stores one alert of each type per trip. Seven AFTER triggers write these records. Two views complete the layer: `v_trip_timeline` (one chronological list of transport, hotel and activity events) and `v_trip_cost_summary` (booked value versus actual spend versus budget per trip).

> **Run order matters.** Script 03 drops the views and audit triggers that script 05 creates, because they depend on the transport tables. After re-running 03 you must re-run 05. Both scripts now drop their own objects first, so every script is genuinely safe to re-run.

5. Sample data

Core schema (script 01) — 312 rows

Table	Rows	Table	Rows
users	6	hotels	8
destinations	7	hotel_amenities	34
places	24	room_types	16
activity_categories	7	rooms	32
expense_categories	6	hotel_reservations	8
amenities	8	hotel_payments	10
trips	5	hotel_reviews	2
trip_members	15	reviews	8
itinerary_days	24	saved_destinations	7
itinerary_items	47	bookings	11
expenses	27	**Total**	**312**

Transport module (script 03) — 87 rows

Table	Rows	Table	Rows
carriers	10	train_details	1
stations	16	cab_details	5
transport_bookings	14	bus_details	3
transport_passengers	33	metro_details	2
flight_details	3	**Total**	**87**

The smart layer adds no seed rows: `audit_log` and `trip_alerts` start empty and fill as data changes. Grand total of seeded rows: **399**.

Trips in the data set

Trip	Dates	Status	Budget (Rs)	Purpose in the demo
Goa Beach Escape	26 to 30 Dec 2026	planned	60,000	Upcoming trip, partial payments, a failed payment, a cancelled reservation
Rajasthan Heritage Tour	10 to 14 Nov 2026	planned	85,000	Two destinations (Jaipur, Udaipur), two hotel reservations, pending bookings
Munnar Monsoon Getaway	14 to 17 Aug 2026	completed	40,000	Finished trip: settlement query and hotel review
Hampi and Coorg Road Trip	19 to 23 Oct 2026	ongoing	35,000	Trip in progress: completed and checked-in stays
Manali Winter Break	8 to 12 Jan 2027	planned	50,000	Cascade-delete demonstration

Other deliberate edge cases: one room under maintenance, one skipped itinerary item, items with no place (SET NULL target), expenses with and without an itinerary link, payments by card, UPI, net banking and cash, a refunded payment, and transport legs for all five modes including cabs without stations.

6. Queries and expected results

The demo file contains 12 queries (Section B), 14 rule tests (Section C) and a procedure demo (Section D). All results below were reproduced on MySQL 8.4.13.

No.	Query	SQL concepts
Q1	Day-wise itinerary of a trip	4-table JOIN, LEFT JOIN, TIME_FORMAT, ORDER BY
Q2	Budget vs actual spend per trip	LEFT JOIN, SUM, GROUP BY, COALESCE, NULLIF
Q3	Spend by expense category per trip	GROUP BY on two columns
Q4	Settlement: who owes whom	Derived tables, CROSS JOIN, arithmetic
Q5	Reservation summary	View with nights, total, paid and balance
Q6	Free rooms for a date range	NOT EXISTS anti-join, user variables
Q7	Hotel performance	Aggregation over a view, LEFT JOIN
Q8	Best-rated destinations	GROUP BY, HAVING, AVG
Q9	Running total of spend	Window function SUM() OVER
Q10	Trip members and roles	GROUP_CONCAT
Q11	Transport or adventure items without booking	LEFT JOIN ... IS NULL
Q12	Owner-is-member integrity check	Outer join as a data audit

The transport and smart layers add three further views used in the demo: `v_trip_transport`, `v_trip_timeline` and `v_trip_cost_summary`.

Expected output: Q2 (budget vs actual)

Trip	Budget	Actual spend	Remaining
Goa Beach Escape	60,000	30,000	30,000
Rajasthan Heritage Tour	85,000	14,600	70,400
Munnar Monsoon Getaway	40,000	30,600	9,400
Hampi and Coorg Road Trip	35,000	22,100	12,900
Manali Winter Break	50,000	5,500	44,500

Expected output: Q4 (settlement, Munnar trip, total 30,600 split three ways = 10,200 each)

Member	Paid	Fair share	Net balance
Sneha Iyer	21,900	10,200	+11,700 (gets back)
Karthik Reddy	6,000	10,200	-4,200 (owes)
Ananya Das	2,700	10,200	-7,500 (owes)

Expected output: Q5 (view v_reservation_summary)

Res.	Hotel, destination	Status	Nights	Total	Paid	Balance
1	Seaside Palms, Goa	confirmed	4	26,000	18,000	8,000
2	Pink City Haveli, Jaipur	confirmed	2	9,600	9,600	0
3	Lake View Palace Inn, Udaipur	pending	2	13,000	2,000	11,000
4	Misty Hills Resort, Munnar	completed	3	13,500	13,500	0
5	Tungabhadra Heritage Stay, Hampi	completed	2	6,400	6,400	0
6	Coffee Mist Homestay, Coorg	checked_in	2	7,600	4,000	3,600
7	Pine Cone Lodge, Manali	pending	4	12,000	3,000	9,000
8	Casa Azul Boutique Stay, Goa	cancelled	4	19,200	0 (refunded)	0

Other expected results: Q6 returns 6 free rooms (Seaside Palms 101 and 202; Casa Azul 101, 102, 201, 202); Q8 shows Goa, Munnar and Hampi tied at 4.5; Q11 returns 6 unbooked transport items; Q12 returns 0 rows.

Rule tests (Section C of the demo file)

Each of these must be rejected: overlapping activities, a day outside the trip, a double-booked room, too many guests, a room under maintenance, a viewer trying to book, a stay outside the trip dates, a review of an unfinished stay, an expense linked to another trip's item, a hotel row for a non-hotel place, rating 6, negative amount, duplicate e-mail, deleting a user who owns trips, and deleting a category in use. Two further tests succeed on purpose and are rolled back: ON DELETE CASCADE (deleting a trip removes its 5 days, 7 items, 2 expenses and 1 reservation) and ON DELETE SET NULL (deleting a place keeps the itinerary item).

7. Implementation plan and work distribution

The plan assumes a team of four. Rename the members and adjust the split if the team size differs.

7.1 Roles

Member	Role	Responsibilities	Deliverables
Member A	Database architect and team lead	Install MySQL and Workbench, run the schema, own scripts 01 and 03, generate and clean the EER diagram, merge changes	Working database, EER diagram image, final schema review
Member B	Data engineer	Review and extend sample data, verify every foreign key, write the data dictionary (table, column, type, constraint, meaning)	Data dictionary, row-count screenshot, data verification notes
Member C	Logic developer	Own the triggers, procedures, the views and the smart layer (05); write and run the 14 rule tests; explain transactions and locking	Trigger documentation, screenshots of each rejected statement, procedure demo
Member D	Query and presentation lead	Own the 12 queries, add 3 queries of the team's own, prepare screenshots, slides and the viva demo script; build the optional Streamlit page	Query report with outputs, slide deck, demo script, optional app

Everyone must be able to explain the whole ER diagram; each member presents the part they own.

7.2 Ten-day schedule

Day	Task	Owner
1	Install MySQL 8.4 LTS and Workbench; create the GitHub repository; run scripts 01, 03 and 05 and confirm 32 / 41 / 31 / 4 in the final quick check	A (all verify)
2	Compare the generated tables with the ER diagram, column by column	A, B
3	Data dictionary; check row counts per table (312 core + 87 transport)	B
4	Walk through every trigger; run rule tests C1 to C14 and capture screenshots	C
5	Run Q1 to Q12; capture results; match them with the expected output in section 6	D
6	Generate the EER diagram (reverse engineer from the script or the live database) and tidy the layout	A
7	Procedure demo (D1 to D5) and view demo; record a short screen capture as a backup	C, D
8	Optional: Streamlit dashboard; team writes 3 extra queries	D, all
9	Compile the report and slides; cross-review each other's sections	All
10	Full rehearsal on a clean machine: run all scripts from scratch	All

7.3 How to run it

1. Open MySQL Workbench and connect to the local server.
2. File, Open SQL Script, choose `01_travel_itinerary_v2_full.sql`, click the lightning bolt (Execute). The last result grid must show tables 21, foreign keys 27, triggers 12, views 1.

3. Open and run `03_transport_module.sql`. The quick check must show tables 30, foreign keys 40, triggers 24, views 2.
4. Open and run `05_smart_features.sql`. The quick check must show tables 32, foreign keys 41, triggers 31, views 4.
5. Open `02_demo_queries.sql`. Run Sections A and B with Ctrl+Enter on each statement. Run Section C one statement at a time, expecting each error. Run Section D in order.
6. For the EER diagram: Database menu, Reverse Engineer, select `travel_itinerary_db`, then File, Export as PNG or PDF. Workbench draws the IS-A subtype as a one-to-one link; label it as IS-A on the exported image.
7. Refresh the schema panel (the refresh icon beside SCHEMAS) if new objects do not appear.

> If you re-run script 03, re-run script 05 afterwards: 03 drops the views and audit triggers that 05 creates. All scripts drop their own objects first, so re-running is safe.

8. Verification status and limitations

> **Verified on MySQL 8.4.13 (26 October 2026).** All scripts were executed on a live server. The acceptance counts matched exactly at every stage: 01 → 21 / 27 / 12 / 1, 03 → 30 / 40 / 24 / 2, 05 → 32 / 41 / 31 / 4. Every script is idempotent (double re-runs produce the same counts). All 14 rule tests in Section C were rejected with the documented errors (SQLSTATE 45000 / 1644, check 3819, unique 1062, foreign key 1451), the CASCADE and SET NULL behaviours worked, and both stored procedures ran correctly (`sp_create_trip` created the trip, the owner membership and one day per date; `sp_book_room` returned a confirmation code and rejected the overlapping re-booking). The transport validation triggers and the audit and budget-alert triggers all fired as designed.

One defect was found and fixed during verification: script 05 did not drop its seven triggers before recreating them, so a second run failed with error 1359 ("Trigger already exists"). The seven `DROP TRIGGER IF EXISTS` statements were added, which is essential because script 03 drops 05's triggers and views.

Limitations: the model has no table for user-uploaded photos or multi-currency support; passwords use a demo hash; the settlement query assumes an equal split among all members including viewers; the audit trail records updates and deletes only, not inserts.

9. Possible extensions

1. Indexes tuned from EXPLAIN output on the three heaviest queries.
2. Role-based MySQL accounts (read-only analyst, trip-editor) using GRANT.
3. A scheduled event that marks trips as ongoing or completed from their dates.
4. Seasonal room pricing and a currency column.
5. Streamlit or Flask front end calling the existing procedures.

10. Conclusion

The Travel Itinerary and Expense Organizer is fully feasible with free tools and a four-person team. The schema, business rules, sample data and the two add-on layers are ready and have now been executed and verified on MySQL 8.4.13. The remaining effort is installation, evidence collection (screenshots, EER diagram) and presentation. We recommend proceeding with MySQL

8.4 LTS and MySQL Workbench as the core stack, and treating the Python and Streamlit layer as an optional enhancement.